游戏助手服务接口说明（Memory 服务）

目录

1 游戏助手服务接口说明（Memory 服务） 3

2 更新日志 3
- 2.1 2026-08-13 . . . 3
- 2.2 2026-07-29 . . . 3
- 2.3 2025-12-18 . . . 3
- 2.4 2025-11-26 . . . 3

3 Memory 服务 3
- 3.1 配置文件 . . . 4
  - 3.1.1 search . . . 4
- 3.2 接口 `/memory/service/enable` . . . 4
  - 3.2.1 Method `Post` . . . 4
  - 3.2.2 Content-Type `application/json` . . . 4
  - 3.2.3 参数 . . . 4
  - 3.2.4 返回 . . . 5
  - 3.2.5 例子 . . . 5
- 3.3 接口 `/memory/service/init` . . . 5
  - 3.3.1 Method `Post` . . . 6
  - 3.3.2 Content-Type `application/json` . . . 6
  - 3.3.3 参数 . . . 6
  - 3.3.4 返回 . . . 6
  - 3.3.5 例子 . . . 6
- 3.4 接口 `/memory/service/list` . . . 6
  - 3.4.1 Method `Get` . . . 6
  - 3.4.2 Content-Type `application/json` . . . 6
  - 3.4.3 参数 . . . 6
  - 3.4.4 返回 . . . 6
  - 3.4.5 例子 . . . 7
- 3.5 接口 `/memory/service/delete` . . . 7
  - 3.5.1 Method `Post` . . . 7
  - 3.5.2 Content-Type `application/json` . . . 7
  - 3.5.3 参数 . . . 7
  - 3.5.4 返回 . . . 7
  - 3.5.5 例子 . . . 7
- 3.6 接口 `/memory/record/list` . . . 7
  - 3.6.1 Method `Post` . . . 7

3.6.2 Content-Type `application/json` . . . 7
3.6.3 参数 . . . 7
3.6.4 返回 . . . 8
3.6.5 例子 . . . 9
3.7 接口 `/memory/record/delete` . . . 10
3.7.1 Method `Post` . . . 10
3.7.2 Content-Type `application/json` . . . 10
3.7.3 参数 . . . 10
3.7.4 返回 . . . 10
3.7.5 例子 . . . 10
3.8 接口 `/memory/record/insert` . . . 11
3.8.1 Method `Post` . . . 11
3.8.2 Content-Type `multipart/form-data` . . . 11
3.8.3 参数 . . . 11
3.8.4 返回 . . . 11
3.8.5 例子 . . . 12
3.9 接口 `/memory/service/search` . . . 13
3.9.1 Method `Post` . . . 13
3.9.2 Content-Type `application/json` . . . 13
3.9.3 参数 . . . 13
3.9.4 返回 . . . 14
3.9.5 例子 . . . 14
3.10 接口 `/memory/service/search/text` . . . 15
3.10.1 Method `Post` . . . 15
3.10.2 Content-Type `application/json` . . . 15
3.10.3 参数 . . . 15
3.10.4 返回 . . . 16
3.10.5 例子 . . . 17
3.11 接口 `/memory/service/search/image` . . . 18
3.11.1 Method `Post` . . . 18
3.11.2 Content-Type `multipart/form-data` . . . 18
3.11.3 参数 . . . 18
3.11.4 返回 . . . 20
3.11.5 例子 . . . 20
3.12 接口 `/memory/service/search/textimage` . . . 22
3.12.1 Method `Post` . . . 22
3.12.2 Content-Type `multipart/form-data` . . . 22
3.12.3 参数 . . . 22
3.12.4 返回 . . . 23
3.12.5 例子 . . . 24
3.13 接口 `/memory/service/build` . . . 26
3.13.1 Method `Post` . . . 26
3.13.2 Content-Type `application/json` . . . 26

3.13.3 参数 26
3.13.4 返回 26
3.13.5 例子 26

1 游戏助手服务接口说明（Memory 服务）

编辑日期 `2026-08-17` 。
游戏助手服务版本 `2026-08-17` 。

2 更新日志

2.1 2026-08-13

- 更新配置文件 `memory.json`
  - 新增 `memory.search.hybrid` 配置，`search_text` 接口检索改为「稠密向量 + BM25 词法 + RRF 融合」混合检索，保留重排序；旧版本保存的内存库自动回退纯稠密检索。

2.2 2026-07-29

- 更新接口 `/memory/record/insert`
  - 增加参数 `mmr_props` ，用于 MMR 查询。
- 添加接口 `/memory/service/search/textimage` ，支持利用 MMR 进行图文对查询、文搜图、图搜图查询。

2.3 2025-12-18

- 更新接口 `/memory/record/list`
  - 增加参数 `indices` ，支持获取指定索引的 Record。
  - 增加参数 `tags` 和 `tags_op` ，支持使用 tag 限定范围。
  - 增加参数 `keys` 、`props` 、`return_image_data` ，支持返回结果中仅包含指定的字段和属性。
- 更新接口 `/memory/service/search` ，增加参数 `keys` 、`props` 、`return_image_data` ，支持返回结果中仅包含指定的字段和属性。
- 更新接口 `/memory/service/search/text` ，增加参数 `keys` 、`props` 、`return_image_data` ，支持返回结果中仅包含指定的字段和属性。
- 更新接口 `/memory/service/search/image` ，增加参数 `keys` 、`props` ，支持返回结果中仅包含指定的字段和属性。

2.4 2025-11-26

- 初版。
- 提供 `Memory 服务` 接口说明。

3 Memory 服务

该服务提供简易的文本存储，支持文本 Embedding 搜索，支持图片存储与搜索。
每个 Instance 即一个实例，代表一个数据库。
每条 Record 即一条记录，以键值对的形式存储，键的类型是字符串，值的类型可以是字符串、数字。针对长文本，可以指定其生成 Embedding 用于搜索。
每条 Record 也可以附上若干张图片，用于图搜图。
支持利用 MMR 服务进行图文对搜索、文搜图、图搜图。
数据库会自动保存至磁盘，自动保存的触发条件详见 `配置文件` 。

3.1 配置文件

配置文件在 `config` 目录下，名为 `memory.json` 。

```
{
    "memory": {
        "enable": true,
        "database": {
            "polling_interval": 1,
            "save_trigger_interval": 15,
            "save_over_changes_count": 100
        },
        "search": {
            "hybrid": {
                "enable": true,
                "bm25": {
                    "tokenizer": "jieba",
                    "k1": 1.5,
                    "b": 0.75
                },
                "rrf_k": 60
            }
        }
    }
}
```

配置项说明：
1. enable：Bool 类型。是否启用 Memory 服务。
2. polling_interval：Int 类型。检测数据库是否需要保存的时间间隔（单位：秒）。
3. save_trigger_interval：Int 类型。自数据库最近一次修改时间算起，经过指定时间间隔（单位：秒）后，保存数据库至磁盘。
4. save_over_changes_count：Int 类型。自数据库最后一次保存时间算起，经过指定修改次数后，保存数据库至磁盘。

3.1.1 search

`search_text` 接口检索相关配置。检索流程：稠密向量 + BM25 词法双路召回 → RRF 融合 → 重排序（如果启用 Rerank）。

- `hybrid.enable` ：Bool 类型。是否启用混合检索（稠密 + BM25 + RRF）。默认 `true` ；关闭后仅使用稠密向量检索。旧版本保存的内存库无 `bm25.pkl` 时会自动回退纯稠密。
- `hybrid.bm25.tokenizer` ：String 类型。BM25 分词方式，可选 `jieba` （默认）或 `bigram` ； `jieba` 不可用时自动回退 `bigram` 。
- `hybrid.bm25.k1` ：Float 类型。BM25 词频饱和参数。默认 1.5。
- `hybrid.bm25.b` ：Float 类型。BM25 文档长度归一化参数。默认 0.75。
- `hybrid.rrf_k` ：Int 类型。RRF（Reciprocal Rank Fusion）融合参数 `k` 。默认 60。

3.2 接口 `/memory/service/enable`

启停或查询 Memory 服务。

3.2.1 Method `Post`

3.2.2 Content-Type `application/json`

3.2.3 参数

1. `enable` ：Bool 类型。 `null` 表示查询， `true` 启用， `false` 停用。

3.2.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "memory": bool 类型。是否启用 Memory 服务。
    }
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。 `enable` 为 `null` 时返回 `data` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

3.2.5 例子

```
# 查询 Memory 服务是否启用

import json
import requests

url = f"http://127.0.0.1:9190/memory/service/enable"
data = { "enable": None }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
""" resp_data
{
    "code": "ok",
    "data": {
        "memory": true
    }
}
"""
```

```
# 启用 Memory 服务

import json
import requests

url = f"http://127.0.0.1:9190/memory/service/enable"
data = { "enable": True }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
""" resp_data
{
    "code": "ok"
}
"""
```

3.3 接口 `/memory/service/init`

创建 Memory 服务实例。

3.3.1 Method `Post`

3.3.2 Content-Type `application/json`

3.3.3 参数

1. `instance_id` : String 类型。实例 ID。

3.3.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` :
1. `ok` : 成功。不会出现 `msg` 。
2. `fail` : 失败。会有 `msg` 给出失败原因。

3.3.5 例子

```
# 创建实例，实例 ID 为`test_instance`

import json
import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/memory/service/init"
data = { "instance_id": instance_id }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

3.4 接口 `/memory/service/list`

列出所有实例。

3.4.1 Method `Get`

3.4.2 Content-Type `application/json`

3.4.3 参数

无

3.4.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "instance_ids": [
            " 实例 ID 列表"
        ]
    }
}
```

返回的 `code` :
1. `ok` : 成功。不会出现 `msg` 。
2. `fail` : 失败。会有 `msg` 给出失败原因。不会出现 `data` 。

3.4.5 例子

```
# 列出所有实例

import requests

url = f"http://127.0.0.1:9190/memory/service/list"
resp = requests.get(url)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"instance_ids": ["test_instance"]}}
```

3.5 接口 /memory/service/delete

删除实例。

3.5.1 Method Post

3.5.2 Content-Type application/json

3.5.3 参数

1. instance_id ：String 类型。实例 ID。

3.5.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 code ：
1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。

3.5.5 例子

```
# 删除实例`test_instance`

import json
import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/memory/service/delete"
data = { "instance_id": instance_id }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

3.6 接口 /memory/record/list

列出指定实例下的所有 Record。

3.6.1 Method Post

3.6.2 Content-Type application/json

3.6.3 参数

1. instance_id ：String 类型。实例 ID。
2. indices ：Int|List[Int] 类型。Record 索引。默认 null，列出所有 Record。

- 当 `indices` 为 Int 类型，且大于等于 0 时，会列出最早更新过的 Record。
- 当 `indices` 为 Int 类型，且小于 0 时，会列出最近更新过的 Record。
- 当 `indices` 为 List[Int] 类型时，会列出指定索引的 Record。

3. `tags` ：List[String] 类型。标签列表。默认 null。通过标签来限定范围。
   - 当 `indices` 为 Int 类型时，会在限定范围后，再列出 Record。
   - 当 `indices` 为 List[Int] 类型时，会取出指定索引的 Record 后，再限定范围。
4. `tags_op` ：String 类型。标签之间的集合操作。下一个标签结果与上一个的集合操作。第一个标签的操作会被忽略，默认 `&&` 。可用的操作如下：
   - `&&` ：两个结果的交集。
   - `||` ：两个结果的并集。
5. `keys` ：List[String] 类型。返回结果中的 Record，仅包含指定的 `keys` 。默认 null，列出所有 `keys` 。当前的 `keys` 有：
   - `record_id` ：String 类型。每条 Record 的 ID 是唯一的。
   - `properties` ：Dict[String, Any] 类型。用户定义的数据，通常以键值对形式保存。
   - `tags` ：List[String] 类型。标签列表，一个 Record 可以有多个标签，方便搜索时过滤。
   - `emb_props` ：List[String] 类型。Record 的 Embedding 属性列表。指定 `properties` 里的键，其对应的值用于生成 Embedding。
   - `images` ：List[Dict[String, Any]] 类型。Record 的图片信息列表。每张图片的信息包含唯一 ID 号、路径、原始文件名、文件大小、文件数据。
   - `mmr_props` ：Dict[String, List[Int]] 类型。Record 的 MMR 属性列表。指定 `properties` 里的键与 `image` 里的索引，其对应的值用于生成图文对 Embedding。
   - `create_time` ：Float 类型。Record 创建时的时间戳，以秒为单位。
   - `update_time` ：Float 类型。Record 最近一次更新的时间戳，以秒为单位。
6. `props` ：List[String] 类型。返回结果中的 Record，仅包含指定的 `properties` 。默认 null，列出所有 `properties` 。
7. `return_image_data` ：Bool 类型。是否返回图片文件。如果传 true 的话，会返回经 base64 编码的图片文件。

格式：

```
{
    "data": {
        "instance_id": "Instance ID",
        "indices": [ 0, 1, 2 ],
        "tags": [ "tag1", "tag2" ],
        "tags_op": "||",
        "keys": [ "key1", "key2" ],
        "props": [ "prop1", "prop2" ],
        "return_image_data": false
    }
}
```

3.6.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "records": [
            "Record 列表"
        ]
    }
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

3.6.5 例子

```
# 列出最近 10 个更新过的 Record,
# 实例 ID 为`test_instance`。

import json
import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/memory/record/list"
data = {
    "instance_id": "inst_1",
    "indices": -3,
    "tags": [
        "event"
    ],
    "tags_op": "&&",
    "keys": [
        "record_id",
        "properties",
        "update_time"
    ],
    "props": [
        "datetime",
        "event",
        "highlight"
    ],
    "return_image_data": False
}
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
""" resp_data
{
  "code": "ok",
  "data": {
    "records": [
        {
            "record_id": "event_47",
            "properties": {
            "datetime": "2025-07-07 14:58:14",
            "event": " 比赛结果：获胜"
            },
            "update_time": 1766045524.5668502
        },
```

```
        {
            "record_id": "event_46",
            "properties": {
            "datetime": "2025-07-07 14:58:03",
            "event": " 回合胜利"
            },
            "update_time": 1766045524.2516127
        },
        {
            "record_id": "event_45",
            "properties": {
            "datetime": "2025-07-07 14:58:03",
            "event": " 开镜爆头击杀 1 名敌人"
            },
            "update_time": 1766045523.94496
        }
    ]
  }
}
"""
```

3.7 接口 /memory/record/delete

从指定实例删除一条 Record。

3.7.1 Method Post

3.7.2 Content-Type application/json

3.7.3 参数

1. instance_id ：String 类型。实例 ID。
2. record_id ：String 类型。Record ID。

3.7.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 code ：
1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。

3.7.5 例子

```
# 删除 Record,
# 实例 ID 为`test_instance`,
# Record ID 为`test_record`

import json
import requests

instance_id = "test_instance"
record_id = "test_record"
url = f"http://127.0.0.1:9190/memory/record/delete"
data = { "instance_id": instance_id, "record_id": record_id }
resp = requests.post(url, data=json.dumps(data))
```

```
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

3.8 接口 /memory/record/insert

插入一条 Record。如果插入的 Record ID 已经存在，之前的数据和图片全部会被删除，替换成当前插入的数据和图片。

3.8.1 Method Post

3.8.2 Content-Type multipart/form-data

3.8.3 参数

1. `data` ：String 类型。JSON 格式的字符串，其内容包含以下参数。
   - `instance_id` ：String 类型。实例 ID。
   - `record_id` ：String 类型。Record ID。
   - `info` ：Dict[String, String|Int|Float] 类型。Record 数据内容，以键值对的形式存储，键的类型是字符串，值的类型可以是字符串、数字。
   - `tags` ：List[String] 类型。标签列表，默认 null。一个 Record 可以有多个标签，方便搜索时过滤。
   - `emb_props` ：List[String] 类型。默认 null。指定需要生成 Embedding 的数据内容，即 `info` 里的键，仅在 Embedding 搜索时用到。
   - `mmr_props` ：Dict[String, List[Int]] 类型。默认 null。指定 `info` 里的键所对应的文本内容或直接填入文本内容，与之对应的 `images` 里索引所指向的图片列表，组成的图文对经过 MMR 生成 Embedding 用于后续的图文搜索。图文搜索请参阅接口 `/memory/service/search/textimage` 。
2. `images` ：List[Byte] 类型。默认 null。图片文件的 Byte 数据。一个 Record 可以有多张图片，仅在图搜图的时候用到。

格式：

```
{
    "data": {
        "instance_id": "Instance ID",
        "record_id": "Record ID",
        "info": { "key1": "value1", "key2": "value2" },
        "tags": [ "tag1", "tag2" ],
        "emb_props": [ "key1", "key2" ],
        "mmr_props": {
            "key1": [ image_index_1, image_index_2 ],
            "key2": [ image_index_2, image_index_3 ],
            "not_key_but_text": [ image_index_3, image_index_4 ]
        }
    }
}
```

3.8.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

3.8.5 例子

```
# 插入一条 Record，
# 实例 ID 为`test_instance`，
# Record ID 为`test_record`。

import json
import requests

info = {
    "datetime": "2025-07-07 14:56:54",
    "event": " 回合胜利",
    "kill": 4,
    "defeat": 0,
    "assistant": 0,
    "weapon": "SG 553",
    "location": " 公寓群",
    "our_team_alive": 4,
    "our_team_death": 1,
    "enemy_team_alive": 0,
    "enemy_team_death": 5,
    "round_time": 51,
    "round": 8,
    "our_score": 7,
    "enemy_score": 1,
    "hp": 77,
    "armor": 99,
    "ammo": 27,
    "clip": 70,
    "money": 10000,
    "highlight": " 在被闪光震撼弹致盲时击杀了一名敌人",
    "rating": " 最多击杀 MVP（4 杀）"
}

images = [
    r"C:\Users\Public\Pictures\0001.jpg",
    r"C:\Users\Public\Pictures\0002.jpg"
]

instance_id = "test_instance"
record_id = "test_record"
url = f"http://127.0.0.1:9190/memory/record/insert"
data = {
    "instance_id": instance_id,
    "record_id": record_id,
    "info": info,
    "tags": [ "round_end", "win" ],
    "emb_props": [ "event", "highlight", "rating" ],
    "mmr_props": {
        "highlight": [0, 1],
        " 闪光弹击杀瞬间": [0]
    }
}
data = {'data': json.dumps(data, ensure_ascii=False)}
files = [ ("images", open(image, 'rb')) for image in images ]
resp = requests.post(url, data=data, files=files)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

3.9 接口 /memory/service/search

给定条件搜索数据库。

3.9.1 Method Post

3.9.2 Content-Type application/json

3.9.3 参数

1. instance_id : String 类型。实例 ID。
2. conditions : List[Dict[String, String]] 类型。查询条件列表，参数如下。
   - key : String 类型。想要查询的 Record 键。
   - value : String|Int|Float 类型。想要匹配的值。
   - op : String 类型。匹配操作。默认 == 。可用的操作如下：
     - == : 数据库值等于 value
     - > : 数据库值大于 value
     - >= : 数据库值大于等于 value
     - < : 数据库值小于 value
     - <= : 数据库值小于等于 value
     - in : 数据库值是否包含 value
     - not in : 数据库值是否不含 value
   - set_op : String 类型。下一个查询条件结果与上一个的集合操作。第一个条件的操作会被忽略，默认 && 。可用的操作如下：
     - && : 两个结果的交集。
     - || : 两个结果的并集。
3. tags : List[String] 类型。标签列表。默认 null。通过标签来限定搜索范围。
   - 不传或者传 null 将在所有 Record 中进行查询。
   - 如果传空列表，将在没有标签的 Record 中进行查询。
   - 如果指定标签，将在拥有该标签的 Record 中进行查询。
4. tags_op : String 类型。标签之间的集合操作。下一个标签结果与上一个的集合操作。第一个标签的操作会被忽略，默认 && 。可用的操作如下：
   - && : 两个结果的交集。
   - || : 两个结果的并集。
5. agg : Dict[String, String] 类型。聚合操作。参数如下。
   - key : String 类型。想要聚合的 Record 键。
   - func : String 类型。聚合操作。可用的操作如下：
     - sum : 对结果进行求和
     - count : 结果计数
     - avg : 对结果求平均
     - min : 值最小的结果
     - max : 值最大的结果
   - group_by : String 类型。分组操作。根据指定的 Record 键进行分组，按组进行聚合操作。默认 null，表示不分组。
   - order : String 类型。根据分组的结果进行排序。默认 null，不排序。可用的排序方式如下：
     - asc : 升序。

  - `desc` ：降序。

6. `keys` ：List[String] 类型。返回结果中的 Record，仅包含指定的 `keys` 。默认 null，列出所有 `keys` 。当前的 `keys` 有：
   - `record_id` ：String 类型。每条 Record 的 ID 是唯一的。
   - `properties` ：Dict[String, Any] 类型。用户定义的数据，通常以键值对形式保存。
   - `tags` ：List[String] 类型。标签列表，一个 Record 可以有多个标签，方便搜索时过滤。
   - `emb_props` ：List[String] 类型。Record 的 Embedding 属性列表。指定 `properties` 里的键，其对应的值用于生成 Embedding。
   - `images` ：List[Dict[String, Any]] 类型。Record 的图片信息列表。每张图片的信息包含唯一 ID 号、路径、原始文件名、文件大小、文件数据。
   - `mmr_props` ：Dict[String, List[Int]] 类型。Record 的 MMR 属性列表。指定 `properties` 里的键与 `image` 里的索引，其对应的值用于生成图文对 Embedding。
   - `create_time` ：Float 类型。Record 创建时的时间戳，以秒为单位。
   - `update_time` ：Float 类型。Record 最近一次更新的时间戳，以秒为单位。
7. `props` ：List[String] 类型。返回结果中的 Record，仅包含指定的 `properties` 。默认 null，列出所有 `properties` 。
8. `return_image_data` ：Bool 类型。是否返回图片文件。如果传 true 的话，会返回经 base64 编码的图片文件。

3.9.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "records": [
            " 查询结果列表"
        ]
    }
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

3.9.5 例子

```
# 查询 Record，统计各区域的 kill 次数。
# 实例 ID 为`test_instance`。

import json
import requests

instance_id = "test_instance"
conditions = [
    { "key": "hp", "value": 0, "op": ">", "set_op": ""},
    { "key": "kill", "value": 0, "op": ">", "set_op": "&&" },
]
```

```
agg = {
    "key": "kill",
    "func": "count",
    "group_by": "location",
    "order": "desc"
}

url = f"http://127.0.0.1:9190/memory/service/search"
data = {
    "instance_id": instance_id,
    "conditions": conditions,
    "tags": None,
    "tags_op": "&&",
    "agg": agg
}
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
""" resp_data
{
    "code": "ok",
    "data": {
        "records": {
            " 炸弹安放区 A": 13,
            " 炸弹安放区 B": 4,
            "CT 起始点": 3,
            " 小路": 3,
            " 公寓群": 3,
            " 丛林": 2,
            " 中路": 2,
            " 连接通道": 2,
            "T 侧斜坡": 1,
            " 商店": 1,
            " 狙击点": 1
        }
    }
}
"""
```

3.10 接口 `/memory/service/search/text`

通过 Embedding 查询 Record。

3.10.1 Method `Post`

3.10.2 Content-Type `application/json`

3.10.3 参数

1. `instance_id` ：String 类型。实例 ID。
2. `text` ：String 类型。查询文本内容。
3. `topk` ：Int 类型。默认 5。仅返回分数最高的 K 个 Record。必须大于 0。
4. `threshold` ：Float 类型。默认 0.5。仅返回分数超过 `threshold` 的 Record。取值范围 `0.0 ~ 1.0` 。
5. `tags` ：List[String] 类型。标签列表。默认 null。通过标签来限定搜索范围。
   - 不传或者传 null 将在所有 Record 中进行查询。

- 如果传空列表，将在没有标签的 Record 中进行查询。
- 如果指定标签，将在拥有该标签的 Record 中进行查询。

6. `tags_op` ：String 类型。标签之间的集合操作。下一个标签结果与上一个的集合操作。第一个标签的操作会被忽略，默认 `&&` 。可用的操作如下：
   - `&&` ：两个结果的交集。
   - `||` ：两个结果的并集。
7. `keys` ：List[String] 类型。返回结果中的 Record，仅包含指定的 `keys` 。默认 null，列出所有 `keys` 。当前的 `keys` 有：
   - `record_id` ：String 类型。每条 Record 的 ID 是唯一的。
   - `properties` ：Dict[String, Any] 类型。用户定义的数据，通常以键值对形式保存。
   - `tags` ：List[String] 类型。标签列表，一个 Record 可以有多个标签，方便搜索时过滤。
   - `emb_props` ：List[String] 类型。Record 的 Embedding 属性列表。指定 `properties` 里的键，其对应的值用于生成 Embedding。
   - `images` ：List[Dict[String, Any]] 类型。Record 的图片信息列表。每张图片的信息包含唯一 ID 号、路径、原始文件名、文件大小、文件数据。
   - `mmr_props` ：Dict[String, List[Int]] 类型。Record 的 MMR 属性列表。指定 `properties` 里的键与 `image` 里的索引，其对应的值用于生成图文对 Embedding。
   - `create_time` ：Float 类型。Record 创建时的时间戳，以秒为单位。
   - `update_time` ：Float 类型。Record 最近一次更新的时间戳，以秒为单位。
8. `props` ：List[String] 类型。返回结果中的 Record，仅包含指定的 `properties` 。默认 null，列出所有 `properties` 。
9. `return_image_data` ：Bool 类型。是否返回图片文件。如果传 true 的话，会返回经 base64 编码的图片文件。

3.10.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "records": [
            " 查询结果列表"
        ]
    }
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

查询结果列表中的 `Record` 格式：

```
{
    "record_id": "Record ID",
    "properties": {
        "key1": "value1",
```

```
        "key2": "value2",
        "keyN": "valueN",
    },
    "tags": [
        "tag1",
        "tag2",
        "tagN"
    ],
    "emb_props": [
        "key1",
        "key2",
        "keyN"
    ],
    "images": [
        {
            "image_id": "Image ID",
            "path": "Image relative path to the `saves` folder",
            "name": "Image name",
            "size": Image Size
        }
    ]
}
```

3.10.5 例子

```
# 查询获胜的 Record。
# 实例 ID 为`test_instance`。

import json
import requests

url = f"http://127.0.0.1:9190/memory/service/search/text"
data = {
    "instance_id": "test_instance",
    "text": " 获胜的回合",
    "topk": 5,
    "threshold": 0.5,
    "tags": None,
    "tags_op": "&&"
}
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
""" resp_data
{
    "code": "ok",
    "data": {
        "records": [
            {
                "record_id": "test_record",
                "properties": {
                    "datetime": "2025-07-07 14:56:54",
                    "event": " 回合胜利",
                    "kill": 4,
                    "defeat": 0,
                    "assistant": 0,
                    "weapon": "SG 553",
                    "location": " 公寓群",
                    "our_team_alive": 4,
```

```
                    "our_team_death": 1,
                    "enemy_team_alive": 0,
                    "enemy_team_death": 5,
                    "round_time": 51,
                    "round": 8,
                    "our_score": 7,
                    "enemy_score": 1,
                    "hp": 77,
                    "armor": 99,
                    "ammo": 27,
                    "clip": 70,
                    "money": 10000,
                    "highlight": " 在被闪光震撼弹致盲时击杀了一名敌人",
                    "rating": " 最多击杀 MVP（4 杀）"
                },
                "tags": [
                    "round_end",
                    "win"
                ],
                "emb_props": [
                    "event",
                    "highlight",
                    "rating"
                ],
                "images": [
                    {
                        "image_id": "30124602a1fdc4a69ee54c1c6d9f1595
                            5733923fff4b47df1e267cf9f651d068",
                        "path": "GATSvrDefaultUser/memory/test_instance/image/
                            36a994ee909d5f23fe72a3f5260bc0b2.jpg",
                        "name": "0001.jpg",
                        "size": 167081
                    },
                    {
                        "image_id": "8304308568cb6cdaba5f73c2dd0f1e2f
                            58b445f5b53fa00fe8a4f4771d09d60a",
                        "path": "GATSvrDefaultUser/memory/test_instance/image/
                            c1c00985be114591ee6d094647b2ee3e.jpg",
                        "name": "0002.jpg",
                        "size": 165291
                    }
                ]
            }
        ]
    }
}
"""
```

3.11 接口 `/memory/service/search/image`

通过图片查询包含相似图片的 Record。

3.11.1 Method `Post`

3.11.2 Content-Type `multipart/form-data`

3.11.3 参数

1. `data` ：String 类型。JSON 格式的字符串，其内容包含以下参数。

- `instance_id` ：String 类型。实例 ID。
- `topk` ：Int 类型。默认 5。仅返回分数最高的 K 个 Record。必须大于 0。
- `threshold` ：Float 类型。默认 0.92。粗略筛选图片的阈值，仅返回分数超过 `threshold` 的 Record。取值范围 `0.0 ~ 1.0` 。
- `threshold_2` ：Float 类型。默认 0.15。进一步筛选图片的阈值。仅返回分数超过 `threshold_2` 的 Record。取值范围 `0.0 ~ 1.0` 。
- `tags` ：List[String] 类型。标签列表。默认 null。通过标签来限定搜索范围。
  - 不传或者传 null 将在所有 Record 中进行查询。
  - 如果传空列表，将在没有标签的 Record 中进行查询。
  - 如果指定标签，将在拥有该标签的 Record 中进行查询。
- `tags_op` ：String 类型。标签之间的集合操作。下一个标签结果与上一个的集合操作。第一个标签的操作会被忽略，默认 `&&` 。可用的操作如下：
  - `&&` ：两个结果的交集。
  - `||` ：两个结果的并集。
- `keys` ：List[String] 类型。返回结果中的 Record，仅包含指定的 `keys` 。默认 null，列出所有 `keys` 。当前的 `keys` 有：
  - `record_id` ：String 类型。每条 Record 的 ID 是唯一的。
  - `properties` ：Dict[String, Any] 类型。用户定义的数据，通常以键值对形式保存。
  - `tags` ：List[String] 类型。标签列表，一个 Record 可以有多个标签，方便搜索时过滤。
  - `emb_props` ：List[String] 类型。Record 的 Embedding 属性列表。指定 `properties` 里的键，其对应的值用于生成 Embedding。
  - `images` ：List[Dict[String, Any]] 类型。Record 的图片信息列表。每张图片的信息包含唯一 ID 号、路径、原始文件名、文件大小、文件数据。
  - `mmr_props` ：Dict[String, List[Int]] 类型。Record 的 MMR 属性列表。指定 `properties` 里的键与 `image` 里的索引，其对应的值用于生成图文对 Embedding。
  - `create_time` ：Float 类型。Record 创建时的时间戳，以秒为单位。
  - `update_time` ：Float 类型。Record 最近一次更新的时间戳，以秒为单位。
- `props` ：List[String] 类型。返回结果中的 Record，仅包含指定的 `properties` 。默认 null，列出所有 `properties` 。
- `return_image_data` ：Bool 类型。是否返回图片文件。如果传 true 的话，会返回经 base64 编码的图片文件。

2. `image` ：Byte 类型。图片文件的 Byte 数据。

格式：

```
{
    "data": {
        "instance_id": "Instance ID",
        "topk": 5,
        "threshold": 0.92,
        "threshold_2": 0.15,
```

```
        "tags": [ "tag1", "tag2" ],
        "tags_op": "&&",
        "return_image_data": false
    }
}
```

3.11.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "records": [
            " 查询结果列表"
        ]
    }
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

查询结果列表中的 `Record` 格式：

```
{
    "record_id": "Record ID",
    "properties": {
        "key1": "value1",
        "key2": "value2",
        "keyN": "valueN",
    },
    "tags": [
        "tag1",
        "tag2",
        "tagN"
    ],
    "emb_props": [
        "key1",
        "key2",
        "keyN"
    ],
    "images": [
        {
            "image_id": "Image ID",
            "path": "Image relative path to the `saves` folder",
            "name": "Image name",
            "size": Image Size,
            "data": "Base64 encoded image file bytes"
        }
    ]
}
```

3.11.5 例子

```
# 查询获胜的 Record。
# 实例 ID 为`test_instance`。
```

```python
import json
import requests

pic_1 = r"C:\Users\Public\Pictures\0001.jpg"

url = f"http://127.0.0.1:9190/memory/service/search/image"
data = {
    "instance_id": "test_instance",
    "topk": 5,
    "threshold": 0.92,
    "threshold_2": 0.15,
    "tags": None,
    "tags_op": "&&",
    "return_image_data": True
}
data = {'data': json.dumps(data, ensure_ascii=False)}
files = [ ("image", open(pic_1, 'rb')) ]
resp = requests.post(url, data=data, files=files)
resp_data = resp.json()
""" resp_data
{
    "code": "ok",
    "data": {
        "records": [
            {
                "record_id": "test_record",
                "properties": {
                    "datetime": "2025-07-07 14:56:54",
                    "event": " 回合胜利",
                    "kill": 4,
                    "defeat": 0,
                    "assistant": 0,
                    "weapon": "SG 553",
                    "location": " 公寓群",
                    "our_team_alive": 4,
                    "our_team_death": 1,
                    "enemy_team_alive": 0,
                    "enemy_team_death": 5,
                    "round_time": 51,
                    "round": 8,
                    "our_score": 7,
                    "enemy_score": 1,
                    "hp": 77,
                    "armor": 99,
                    "ammo": 27,
                    "clip": 70,
                    "money": 10000,
                    "highlight": " 在被闪光震撼弹致盲时击杀了一名敌人",
                    "rating": " 最多击杀 MVP（4 杀）"
                },
                "tags": [
                    "round_end",
                    "win"
                ],
                "emb_props": [
                    "event",
                    "highlight",
                    "rating"
```

```
                ],
                "images": [
                    {
                        "image_id": "30124602a1fdc4a69ee54c1c6d9f1595
                            5733923fff4b47df1e267cf9f651d068",
                        "path": "GATSvrDefaultUser/memory/test_instance/image/
                            36a994ee909d5f23fe72a3f5260bc0b2.jpg",
                        "name": "0001.jpg",
                        "size": 167081,
                        "data": "/9j/4AAQS...QO+LQ//9k="
                    },
                    {
                        "image_id": "8304308568cb6cdaba5f73c2dd0f1e2f
                            58b445f5b53fa00fe8a4f4771d09d60a",
                        "path": "GATSvrDefaultUser/memory/test_instance/image/
                            c1c00985be114591ee6d094647b2ee3e.jpg",
                        "name": "0002.jpg",
                        "size": 165291,
                        "data": "/9j/4AAQS...7dINm4cf/Z"
                    }
                ]
            }
        ]
    }
}
"""
```

3.12 接口 `/memory/service/search/textimage`

通过图文查询包含相似图片和描述的 Record。

可以只用简短的描述来查询，亦可只用图片来查询，亦或是文本加图片一起查询。

3.12.1 Method `Post`

3.12.2 Content-Type `multipart/form-data`

3.12.3 参数

1. `data` ：String 类型。JSON 格式的字符串，其内容包含以下参数。
   - `instance_id` ：String 类型。实例 ID。
   - `text` ：String 类型。默认空字符串。描述文本。与 `image` 参数二选一传入，或二者同时传入。
   - `topk` ：Int 类型。默认 5。仅返回分数最高的 K 个 Record。必须大于 0。
   - `threshold` ：Float 类型。默认 0.2。筛选阈值，仅返回分数超过 `threshold` 的 Record。取值范围 `0.0 ~ 1.0` 。
   - `tags` ：List[String] 类型。标签列表。默认 null。通过标签来限定搜索范围。
     - 不传或者传 null 将在所有 Record 中进行查询。
     - 如果传空列表，将在没有标签的 Record 中进行查询。
     - 如果指定标签，将在拥有该标签的 Record 中进行查询。
   - `tags_op` ：String 类型。标签之间的集合操作。下一个标签结果与上一个的集合操作。第一个标签的操作会被忽略，默认 `&&` 。可用的操作如下：
     - `&&` ：两个结果的交集。

  - `||` ：两个结果的并集。

- `keys` ：List[String] 类型。返回结果中的 Record，仅包含指定的 `keys` 。默认 null，列出所有 `keys` 。当前的 `keys` 有：
  - `record_id` ：String 类型。每条 Record 的 ID 是唯一的。
  - `properties` ：Dict[String, Any] 类型。用户定义的数据，通常以键值对形式保存。
  - `tags` ：List[String] 类型。标签列表，一个 Record 可以有多个标签，方便搜索时过滤。
  - `emb_props` ：List[String] 类型。Record 的 Embedding 属性列表。指定 `properties` 里的键，其对应的值用于生成 Embedding。
  - `images` ：List[Dict[String, Any]] 类型。Record 的图片信息列表。每张图片的信息包含唯一 ID 号、路径、原始文件名、文件大小、文件数据。
  - `mmr_props` ：Dict[String, List[Int]] 类型。Record 的 MMR 属性列表。指定 `properties` 里的键与 `image` 里的索引，其对应的值用于生成图文对 Embedding。
  - `create_time` ：Float 类型。Record 创建时的时间戳，以秒为单位。
  - `update_time` ：Float 类型。Record 最近一次更新的时间戳，以秒为单位。
- `props` ：List[String] 类型。返回结果中的 Record，仅包含指定的 `properties` 。默认 null，列出所有 `properties` 。
- `return_image_data` ：Bool 类型。是否返回图片文件。如果传 true 的话，会返回经 base64 编码的图片文件。

2. `image` ：Byte 类型。图片文件的 Byte 数据。与 `text` 参数二选一传入，或二者同时传入。

格式：

```
{
    "data": {
        "instance_id": "Instance ID",
        "text": "short description of the image",
        "topk": 5,
        "threshold": 0.2,
        "tags": [ "tag1", "tag2" ],
        "tags_op": "&&",
        "return_image_data": false
    }
}
```

3.12.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "records": [
            " 查询结果列表"
        ]
    }
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。

2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

查询结果列表中的 `Record` 格式：

```
{
    "record_id": "Record ID",
    "properties": {
        "key1": "value1",
        "key2": "value2",
        "keyN": "valueN",
    },
    "tags": [
        "tag1",
        "tag2",
        "tagN"
    ],
    "emb_props": [
        "key1",
        "key2",
        "keyN"
    ]
    "images": [
        {
            "image_id": "Image ID",
            "path": "Image relative path to the `saves` folder",
            "name": "Image name",
            "size": Image Size,
            "data": "Base64 encoded image file bytes"
        }
    ],
    "mmr_props": {
         "key1": [ image_index_1, image_index_2 ],
         "key2": [ image_index_2, image_index_3 ],
         "not_key_but_text": [ image_index_3, image_index_4 ]
    }
}
```

3.12.5 例子

```
# 查询使用闪光弹击杀的 Record。
# 实例 ID 为`test_instance`。

import json
import requests

pic_1 = r"C:\Users\Public\Pictures\0001.jpg"

url = f"http://127.0.0.1:9190/memory/service/search/textimage"
data = {
    "instance_id": "test_instance",
    "text": " 闪光击杀",
    "topk": 5,
    "threshold": 0.2,
    "tags": None,
    "tags_op": "&&",
    "return_image_data": True
}
data = {'data': json.dumps(data, ensure_ascii=False)}
files = [ ("image", open(pic_1, 'rb')) ]
```

```
resp = requests.post(url, data=data, files=files)
resp_data = resp.json()
""" resp_data
{
    "code": "ok",
    "data": {
        "records": [
            {
                "record_id": "test_record",
                "properties": {
                    "datetime": "2025-07-07 14:56:54",
                    "event": " 回合胜利",
                    "kill": 4,
                    "defeat": 0,
                    "assistant": 0,
                    "weapon": "SG 553",
                    "location": " 公寓群",
                    "our_team_alive": 4,
                    "our_team_death": 1,
                    "enemy_team_alive": 0,
                    "enemy_team_death": 5,
                    "round_time": 51,
                    "round": 8,
                    "our_score": 7,
                    "enemy_score": 1,
                    "hp": 77,
                    "armor": 99,
                    "ammo": 27,
                    "clip": 70,
                    "money": 10000,
                    "highlight": " 在被闪光震撼弹致盲时击杀了一名敌人",
                    "rating": " 最多击杀 MVP（4 杀）"
                },
                "tags": [
                    "round_end",
                    "win"
                ],
                "emb_props": [
                    "event",
                    "highlight",
                    "rating"
                ],
                "images": [
                    {
                        "image_id": "30124602a1fdc4a69ee54c1c6d9f1595
                            5733923fff4b47df1e267cf9f651d068",
                        "path": "GATSvrDefaultUser/memory/test_instance/image/
                            36a994ee909d5f23fe72a3f5260bc0b2.jpg",
                        "name": "0001.jpg",
                        "size": 167081,
                        "data": "/9j/4AAQS...QO+LQ//9k="
                    },
                    {
                        "image_id": "8304308568cb6cdaba5f73c2dd0f1e2f
                            58b445f5b53fa00fe8a4f4771d09d60a",
                        "path": "GATSvrDefaultUser/memory/test_instance/image/
                            c1c00985be114591ee6d094647b2ee3e.jpg",
                        "name": "0002.jpg",
```

```
                    "size": 165291,
                    "data": "/9j/4AAQS...7dINm4cf/Z"
                }
            ],
            "mmr_props": {
                "highlight": [
                    0,
                    1
                ],
                " 闪光弹击杀瞬间": [
                    0
                ]
            },
            "create_time": 1785305039.7976034,
            "update_time": 1785306050.579238
        }
    ]
  }
}
"""
```

3.13 接口 /memory/service/build

将实例数据库保存至磁盘。
如果不想等待自动保存，可以调用此接口立即保存数据库。

3.13.1 Method Post

3.13.2 Content-Type application/json

3.13.3 参数

1. instance_id ：String 类型。实例 ID。

3.13.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 code ：
1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。

3.13.5 例子

```
# 保存实例数据库

import json
import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/memory/service/build"
data = { "instance_id": instance_id }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
# resp_data: {"code": "ok"}
```